ШИНЖЛЭХ УХААН ТЕХНОЛОГИЙН ИХ СУРГУУЛЬ
Мэдээлэл холбооны технологийн сургууль



БИЕ ДААЛТ 1 ТАЙЛАН

F.CSM302 ҮЙЛДЛИЙН СИСТЕМ

Бие даалтын ажлын нэр:	The Sleeping Teaching Assistant
Хичээл заасан багш:	Г.Ганбат
Багийн дугаар:	II-4-10
Багийн гишүүд:	Оюутан 1: А.Төгөлдөр B242270001
	Оюутан 2: Ц.Цогтжаргал B241940005
	Оюутан 3: Д.Отгон-Эрдэнэ B222270083
	Оюутан 4: О.Цэрэнлхавга B232270113
	Оюутан 5: Г.Түвшинсайхан B242270085

Улаанбаатар хот, 2026 он

Гарчгийн жагсаалт

Хураангуй 4
1. Удиртгал 4
1.1 Сэдвийн холбогдол 4
1.2 Асуудлын тодорхойлолт 4
1.3 Зорилго, зорилт 4
1.4 Хамрах хүрээ 5
1.5 Core-ийн Synchronization объектууд 5
2. Сурах бичгийн шаардлага 5
3. Онолын үндэс 6
3.1 Race condition ба critical section 6
3.2 Mutex lock 6
3.3 Semaphore 6
3.4 Bounded-buffer бодлоготой ижил хэв маяг 6
3.5 Deadlock ба starvation 7
4. Системийн зохиомж 7
4.1 Давхарга ба компонентууд 7
4.2 Өгөгдлийн урсгал 8
4.3 TA-ийн төлөвийн диаграм 8
4.4 Event-ийн төрлүүд 9
5. Хэрэгжилт 9
5.1 Хэл, сан, орчин 9
5.2 Тохиргоо ба оролтын шалгалт 9
5.3 TA thread 9
5.4 Оюутны thread 10
5.5 Чухал шийдвэрүүд 10
5.6 Удирдлагын функцүүд 10
5.7 UI-ийн хэрэгжилт 11
5.8 AI болон гадны материал ашигласан тухай 11
6. Туршилт ба UI дүрслэл 11
6.1 Юуг өөрчилж турших вэ 11
6.2 Удирдлага ба ажиглалтын хэрэгсэл 11
6.3 Сценариуд 12
7. Туршилтын арга 12
7.1 Орчин 12
7.2 Гурван түвшний туршилт 12
7.3 Өгөгдөл ба хүлээгдэх үр дүн 12
7.4 Сценарийн тодорхойлолт 12
8. Үр дүн 13
8.1 Автомат тестийн үр дүн 13
8.2 UI сценарийн үр дүн 13
8.3 UI-ийн тоо ба core-ийн тоо 16
9. Хэлэлцүүлэг 16
9.1 UI-ийн дүрслэл ачаалал ихтэй үед хоцрох 16

9.2 STEP товчны ажиллагаа 17
9.3 Бусад хязгаарлалт 17
9.4 Synchronization-ы зөв байдлын тайлбар 17
10. Дүгнэлт 17
Ашигласан материал 18
Хавсралт А. Ажлын төлөвлөгөө 19

Хураангуй

Энэхүү бие даалтаар Silberschatz-ийн Operating System Concepts (10th edition) сурах бичгийн 7-р бүлгийн "The Sleeping Teaching Assistant" програмчлалын төслийг Python хэл дээр хэрэгжүүлэв. Бодлогод нэг туслах багш (TA), хязгаарлагдмал тооны хүлээлгийн сандал, олон оюутан оролцоно. Оюутан байхгүй үед TA унтаж, оюутан ирэхэд сэрнэ. Сандал дүүрэн үед ирсэн оюутан буцна.

Төслийг хоёр хэсэгт хуваасан. Core (`ta_core.py`) нь TA болон оюутан бүрийг бодит thread болгон ажиллуулж, хүлээлгийн дарааллыг Lock (mutex)-ээр хамгаалж, унтах ба сэрэхийг Semaphore-оор хэрэгжүүлсэн. Core нь UI-аас хамааралгүй бөгөөд thread бүр өөрийн үйлдлийг event болгон дарааламд (`queue.Queue`) хийдэг. UI (`ui.py`) нь Tkinter дээр бүтсэн dashboard бөгөөд тэр event-үүдийг уншиж TA, сандал, оюутныг хөдөлгөөнт дүрслэлээр, мөн тоолуур, график, event түүхээр бодит цагт харуулна. Хэрэглэгч сандлын тоо, оюутны тоо, ирэх давтамж, үйлчилгээний хугацааг тохируулж, START, PAUSE, STEP, RESET удирдлагаар симуляцийг ажиглана.

Туршилтыг гурван түвшинд хийсэн: багийн 11 автомат тест, 27 нэмэлт edge case тест, 11 UI сценари. Бүх тест тэнцсэн. Synchronization-ы гол шинж (дарааламал сандлын тооноос хэтрэхгүй, TA нэг дор нэг оюутанд үйлчилнэ, FIFO дарааламал, deadlock-гүй) бүх тохиргоонд зөв ажилласан. Ачаалал ихтэй тохиргоонд UI-ийн дүрслэл core-ийн бодит тооноос хоцорч харагддаг нь гол хязгаарлалт болохыг тогтоов.

1. Удиртгал

1.1 Сэдвийн холбогдол

Орчин үеийн үйлдлийн систем олон процесс, thread-ийг зэрэг ажиллуулдаг. Эдгээр нь нийтийн нөөц (санах ой, дарааламал, төхөөрөмж)-ийг хуваалцах үед синхрончлол алдагдвал race condition, deadlock зэрэг ноцтой алдаа гарна. Тиймээс mutex ба semaphore-оор нөөцийг зөв хамгаалж, thread-үүдийг зөв унтуулж сэрээх нь үйлдлийн системийн гол ойлголтуудын нэг юм.

1.2 Асуудлын тодорхойлолт

- TA-ийн өрөөнд нэг сандал (TA), коридорт N хүлээлгийн сандал байна.
- Оюутан байхгүй бол TA унтана.
- Оюутан ирэхэд TA унтаж байвал сэрээнэ. TA завгүй бол сул сандалд суугаад хүлээнэ. Сул сандал байхгүй бол буцаж явна, хэсэг хугацааны дараа дахин ирнэ.
- Оюутнууд FIFO дарааллаар үйлчлүүлнэ.

1.3 Зорилго, зорилт

Зорилго: Sleeping Teaching Assistant бодлогыг thread, semaphore, mutex ашиглан зөв хэрэгжүүлж, ажиллагааг нь харагдахуйц дүрслэх.

Зорилтууд:

- UI-аас хамааралгүй, туршиж болохуйц core симуляци бичих (`ta_core.py`).

- Core-ийн бодит thread-үүдийн төлөвийг харуулсан интерактив Tkinter dashboard хийх (`ui.py`).
- Оролтыг шалгаж, буруу утгыг ойлгомжтой мессежээр мэдэгдэх.
- START, PAUSE, STEP, RESET удирдлага, event түүхийг шүүх, хайх, CSV болгон хадгалах боломж олгох.
- Автомат тест, edge case тест, UI сценариар зөв байдлыг баталгаажуулах.

1.4 Хамрах хүрээ

Төсөл нь нэг TA-тай систем, оюутан бүрийг тусдаа thread болгон загварчилсан симуляцийг хамарна. Хэрэглэгч сандлын тоо (0–20), оюутны тоо (1–50), ирэх болон үйлчлэх дундаж хугацаа (0.001–60 сек)-г тохируулна. Төсөл нь олон TA, оюутны ээлжийн давуу эрх, процесс хоорондын (multi-process) synchronization-ыг хамаарахгүй.

1.5 Core-ийн Synchronization объектууд

Объект	Төрөл	Үүрэг
`mutex`	`threading.Lock`	Хүлээлгийн дараалал, TA-ийн төлөв, тоолуурыг хамгаална
`students_sem`	`Semaphore(0)`	Хүлээж буй оюутны тоо. TA үүн дээр wait хийж унтана, оюутан signal өгч сэрээнэ
`turn[i]`	`Semaphore(0)`	i-р оюутны хувийн semaphore. TA үйлчилж дуустал оюутан үүн дээр хүлээнэ
`_run`	`threading.Event`	Set = ажиллаж байна, clear = түр зогссон (pause)
`_stop`	`threading.Event`	Бүх thread-ийг зогсоох дохио

2. Сурах бичгийн шаардлага

Төсөл нь Silberschatz нарын *Operating System Concepts* (10th edition) номын 7-р бүлгийн Programming Project 10 дээр суурилсан [1]. Номын даалгаврын агуулгыг дараах байдлаар товчилж болно.

- Их сургуулийн компьютер судлалын тэнхимд оюутнуудад програмчлалын даалгавар дээр тусалдаг туслах багш (TA) байна.
- TA-ийн өрөө жижиг тул зөвхөн нэг ширээ, нэг сандалтай. Өрөөний гадна коридорт гурван сандал байх бөгөөд TA өөр оюутанд тусалж байгаа үед оюутан тэнд суугаад хүлээнэ.
- Тусламж хүсэх оюутан байхгүй үед TA ширээн дээрээ унтана.
- Оюутан ирээд TA унтаж байгааг олвол TA-г сэрээж тусламж хүснэ. TA өөр оюутанд тусалж байвал оюутан коридорын сандалд суугаад хүлээнэ. Сандал бүгд дүүрэн бол оюутан дараа дахин ирнэ.
- Даалгавар нь TA болон оюутнуудын үйл ажиллагааг mutex lock, semaphore ашиглан зохицуулсан шийдэл бичихийг шаарддаг. Ном POSIX thread (C хэл) ашиглахыг санал болгосон.

Бидний хэрэгжүүлэлт ба номын зөрүү. Бид Python-ийн `threading` санг ашигласан. `threading.Lock` нь mutex lock, `threading.Semaphore` нь POSIX semaphore-тэй адил үүрэгтэй (`acquire()` нь `wait()`, `release()` нь `signal()`). Номд хүлээлгийн сандал 3 байдаг бол бид үүнийг тохируулагддаг параметр болгосон.

Хичээлийн нэмэлт шаардлага (queue animation). Сандлын тоо, оюутны ирэх давтамж, үйлчилгээний хугацааг тохируулж, хүлээлгийн дараалал болон TA-ийн төлөвийг хөдөлгөөнт эсвэл алхамчилсан байдлаар харуулах интерактив UI хийх. Мөн UI нь бодит thread-ийн төлөвтэй нийцэх, буруу оролтод ойлгомжтой мэдээлэл өгөх, тохиргоог солих, reset хийх боломжтой байх.

3. Онолын үндэс

3.1 Race condition ба critical section

Хэд хэдэн thread нийтийн өгөгдлийг зэрэг уншиж бичихэд үр дүн нь thread-үүдийн ажиллах дарааллаас хамаарвал **race condition** үүснэ. Нийтийн өгөгдөл рүү хандаж буй кодын хэсгийг **critical section** гэнэ. Нэг мөчид зөвхөн нэг thread critical section-д байхыг баталгаажуулах нь хамгийн энгийн хамгаалалт юм [1].

Манай бодлогод нийтийн өгөгдөл нь хүлээлгийн дараалал (`waiting`), сул сандлын тоо, TA-ийн төлөв, тоолуурууд юм. Хоёр оюутан нэгэн зэрэг сүүлийн сул сандлыг авахыг оролдвол mutex-гүй үед хоёулаа "сул сандал байна" гэж үзээд дараалал сандлын тооноос хэтэрч болно.

3.2 Mutex lock

Mutex lock нь critical section-д орохын өмнө `acquire()`, гарсны дараа `release()` хийдэг. Түгжээ авсан thread нэг л байх тул нэгэн зэрэг өөрчлөлт хийх боломжгүй. Python-д `threading.Lock` үүнийг хэрэгжүүлнэ.

3.3 Semaphore

Semaphore нь бүхэл тоон утгатай, зөвхөн `wait()` ба `signal()` үйлдлээр өөрчлөгддөг synchronization хэрэгсэл юм. `wait()` нь утгыг 1-ээр бууруулж, хэрэв утга 0 байвал thread-ийг блоклоно. `signal()` нь утгыг 1-ээр нэмж, блоклогдсон thread байвал сэрээнэ. Python-д эдгээр нь `acquire()` ба `release()`. Semaphore-ийг хоёр зорилгоор ашиглана [1]:

- **Тоолох:** нөөцийн тоог тоолно. Манай `students_sem` нь хүлээж буй оюутны тоог тоолж, 0 болоход TA блоклогдон унтана.
- **Дохио өгөх:** нэг thread нөгөөг сэрээнэ. Манай `turn[i]` нь TA үйлчилж дуусахад i-р оюутанд дохио өгнө.

3.4 Bounded-buffer бодлоготой ижил хэв маяг

7-р бүлгийн bounded-buffer бодлого нь `mutex`, `empty`, `full` гурван semaphore ашигладаг: producer `wait(empty)`, `wait(mutex)`-ээр буферт нэмж, `signal(mutex)`, `signal(full)` хийнэ. Consumer нь эсрэг дарааллаар ажиллана [1]. Sleeping TA бодлого энэтэй төстэй: оюутнууд producer (дараалалд орно), TA consumer (дараалалаас авна), хүлээлгийн сандлууд

хязгаарлагдмал буфер юм. Ялгаа нь буфер дүүрэн үед producer хүлээхгүй, **буцаж явдагт** оршино.

Bounded-buffer	**Sleeping TA**
Producer	Оюутан (дараалалд орно)
Consumer	TA (дараалалаас авч үйлчилнэ)
Буфер n слот	N хүлээлгийн сандал
`full` semaphore	`students_sem` (хүлээж буй оюутны тоо)
`mutex`	`mutex`
Буфер дүүрэн үед producer хүлээнэ	Сандал дүүрэн үед оюутан буцна (`turned_away`)

3.5 Deadlock ба starvation

Deadlock нь thread-үүд бие биеийнхээ барьсан нөөцийг хүлээж, хэзээ ч урагшлахгүй байх байдал. Манай шийдэлд deadlock-оос хамгаалах гол арга нь **mutex барьсан үедээ semaphore дээр блоклогддоггүй** явдал юм. TA `students_sem.acquire()`-ийг mutex-ийн гадна хийдэг, оюутан `turn[i].acquire()`-ийг mutex-ийн гадна хийдэг. Нэг л mutex байдаг тул түгжээний дарааллын асуудал гарахгүй.

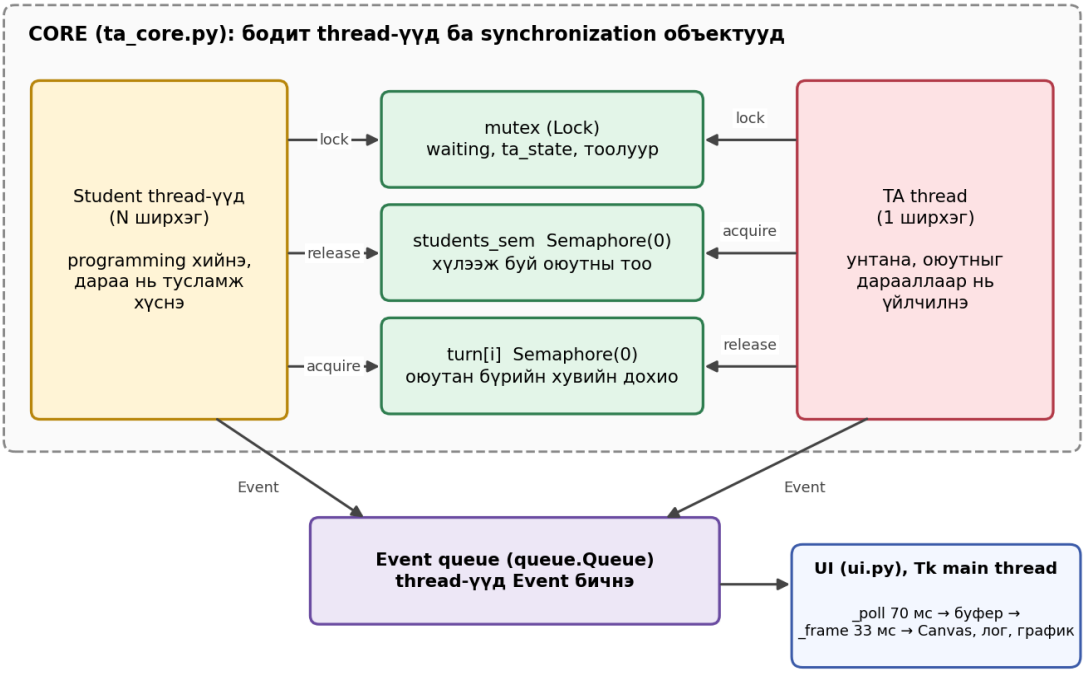
Starvation нь нэг thread хэзээ ч нөөц авч чадахгүй байх байдал. Бид оюутныг `deque` дээр FIFO дарааллаар хадгалж, TA толгойгоос нь авдаг тул дараалалд орсон оюутан заавал үйлчлүүлнэ. Харин сандал дүүрэн үед буцсан оюутан дахин ирэхэд суух баталгаа байхгүй (энэ нь бодлогын нөхцөл).

4. Системийн зохиомж

4.1 Давхарга ба компонентууд

Систем нь хоёр давхаргатай. Core нь бүх concurrency-ийг хариуцаж, UI нь зөвхөн core-оос ирсэн event-үүдийг харуулна. Ингэснээр core-ийг UI-гүйгээр (консол, автомат тест) шалгах боломжтой болсон.

Компонент	**Файл**	**Үүрэг**
`Config`	`src/ta_core.py`	Тохиргооны өгөгдөл ба шалгалт (`validate`, `from_text`). Буруу утганд `ValueError` шиднэ
`Simulation`	`src/ta_core.py`	TA ба оюутны thread үүсгэх, `start`, `pause`, `resume`, `stop`, `reset`, `drain_events`
`Event`	`src/ta_core.py`	Нэг үйл явдал ба тухайн мөчийн төлөвийн зураг (snapshot). Өөрчлөгдөхгүй (`frozen`) dataclass
`Dashboard`	`src/ui.py`	Tkinter UI: тохиргоо, удирдлага, canvas, график, лог
Автомат тест	`src/test_ta_core.py`, `tests/`	Unit, edge case, сценарийн тест



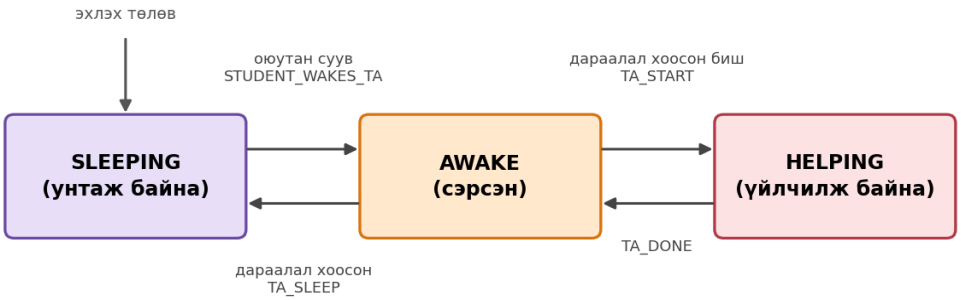
Зураг 4.1. Core ба UI-ийн бүтэц, synchronization объектууд ба өгөгдлийн урсгал

4.2 Өгөгдлийн урсгал

1. Thread (TA эсвэл оюутан) үйлдэл хийхдээ `mutex` авч, төлвөө өөрчилнө.
2. Төлөв өөрчлөгдсөн даруйдаа, **mutex-ээ барьсан хэвээр**, `Event` үүсгэж `queue.Queue`-д хийнэ. Event нь тухайн мөчийн TA-ийн төлөв, дараалал, үйлчлүүлж буй оюутан, тоолуурыг агуулна.
3. UI-ийн `_poll` (70 мс тутам) queue-г уншиж, event-үүдийг буферт хийнэ.
4. UI-ийн `_frame` (33 мс тутам) буферээс event-үүдийг дараалаар нь авч, canvas, тоолуур, график, логийг шинэчилнэ.

Event-ийг mutex барьсан үед үүсгэдэг тул queue дахь event-үүдийн дараалал нь бодит төлвийн өөрчлөлтийн дараалалтай яг таарна. Tkinter-ийн виджетийг зөвхөн үндсэн thread шинэчилдэг тул thread-safety-ийн асуудал гарахгүй.

4.3 TA-ийн төлөвийн диаграм



Зураг 4.2. TA-ийн төлвүүд ба шилжилт (`SLEEPING`, `AWAKE`, `HELPING`)

TA эхэндээ SLEEPING төлөвт байна. Оюутан сандалд суухдаа TA унтаж байвал төлвийг AWAKE болгож, STUDENT_WAKES_TA event үүсгэнэ. TA дараалын эхний оюутныг авч HELPING болно (TA_START). Үйлчилж дуусаад AWAKE болно (TA_DONE). Дараалал хоосон бол SLEEPING болно (TA_SLEEP).

4.4 Event-ийн төрлүүд

Event	Хэзээ үүсэх вэ
SIM_START, SIM_STOP	Симуляци эхэлсэн, зогссон
STUDENT_WAKES_TA	Оюутан унтаж буй TA-г сэрээв
STUDENT_WAITS	Оюутан хүлээлгийн сандалд суув
STUDENT_LEAVES	Сандал дүүрэн тул оюутан буцав
TA_START	TA оюутанд үйлчилж эхлэв
TA_DONE	TA үйлчилж дууслаа
TA_SLEEP	Дараалал хоосорсон тул TA унтлаа

5. Хэрэгжилт

5.1 Хэл, сан, орчин

Төслийг Python 3.14 дээр бичсэн. Зөвхөн стандарт сан ашигласан: threading (thread, Lock, Semaphore, Event), queue (thread хоорондын event дамжуулалт), collections.deque (FIFO дараалал), dataclasses, tkinter (UI), unittest (тест). Гадны сан ашиглаагүй.

5.2 Тохиргоо ба оролтын шалгалт

Config dataclass нь num_chairs, num_students, arrival_interval, service_time ба заавал биш seed талбартай. validate() нь утгыг шалгаж, буруу бол ойлгомжтой мессежтэй ValueError шиднэ. from_text() нь UI-ийн текст талбаруудыг хөрвүүлж, бүхэл тоо байх талбарт бутархай, текст, хоосон утга орвол алдаа өгнө. Хязгаарууд нь MAX_CHAIRS = 20, MAX_STUDENTS = 50, MAX_SECONDS = 60 гэсэн тогтмолууд бөгөөд кодод тархаагүй. nan, inf-ийг хугацааны шалгалт татгалзана.

5.3 TA thread

TA thread нь төгсгөлгүй давтана: дараалал хоосон бол унтана, students_sem дээр блоклогдоно, сэрэхэд эхний оюутныг авч үйлчилнэ, дуусмагц тухайн оюутанд дохио өгнө. Гол хэсгийг доор үзүүлэв (message-үүдийг хасаж хураангуйлсан).

```
def _ta_loop(self):
    while not self._stop.is_set():
        with self.mutex:  # 1) хүн байхгүй бол унтна
            if not self.waiting and self.ta_state != TA_SLEEPING:
                self.ta_state = TA_SLEEPING
                self._emit(TA_SLEEP, None, ...)
        got = False  # 2) wait(students): блоклогдоно
        while not self._stop.is_set() and not got:
            got = self.students_sem.acquire(timeout=0.05)
        if not got: break
        with self.mutex:  # 3) эхний оюутныг авна
```

```
            sid = self.waiting.popleft()
            self.current, self.ta_state = sid, TA_HELPING
            self._emit(TA_START, sid, ...)
        self._sleep(self._around(rng, service_time))  # 4) үйлчилнэ
        with self.mutex:  # 5) дуусгана
            self.served += 1
            self.current, self.ta_state = None, TA_AWAKE
            self._emit(TA_DONE, sid, ...)
        self.turn[sid].release()  #    signal(turn[sid])
```

5.4 Оюутны thread

```
def _student_loop(self, sid, rng):
    while not self._stop.is_set():
        self._sleep(self._around(rng, arrival_interval))  # programming хийнэ
        seated = False
        with self.mutex:
            if len(self.waiting) < num_chairs:  # сул сандал байна уу?
                if self.ta_state == TA_SLEEPING:
                    self.ta_state = TA_AWAKE  # TA-г сэрээнэ
                    self._emit(STUDENT_WAKES_TA, sid, ...)
                self.waiting.append(sid)
                self._emit(STUDENT_WAITS, sid, ...)
                self.students_sem.release()  # signal(students)
                seated = True
            else:
                self.turned_away += 1  # сандал дүүрэн, буцна
                self._emit(STUDENT_LEAVES, sid, ...)
        if seated:  # ээлжээ хүлээнэ (mutex-гүй)
            while not self._stop.is_set():
                if self.turn[sid].acquire(timeout=0.05): break
```

5.5 Чухал шийдвэрүүд

Шийдвэр	Шалтгаан
Оюутан бүрт хувийн `turn[i]` semaphore	Номын нэг нийтийн semaphore-оор дохио өгвөл хүлээж буй оюутнуудаас аль нь сэрэх нь тодорхойгүй болж, UI дээрх дараалал бодит дарааллаас зөрнө. Хувийн semaphore нь TA-ийн сонгосон оюутныг яг сэрээнэ
Event-ийг mutex барьсан үед үүсгэх	Event-ийн дараалал төлвийн өөрчлөлтийн дарааллаар таарна. `queue.Queue.put` нь блоклодоггүй тул deadlock үүсгэхгүй
Semaphore дээр блоклох үед mutex-ийг барихгүй	Deadlock-оос сэргийлнэ (3.5)
`acquire(timeout=0.05)` давталт	Thread нь `stop` дохиог 50 мс-ээс хоцроохгүй мэдэрч, цэвэр зогсоно. Pause хийсэн үед ч гацахгүй
`_sleep()` нь pause-г хүндэтгэнэ	Pause үед хугацаа хасагдахгүй, үргэлжлүүлэхэд үлдсэнээс нь явна
Хугацааг дунджийн 50%–150% хооронд санамсаргүй болгох	Бодит байдалд ойр, thread-үүд нэгэн зэрэг ирэхээс сэргийлнэ. `seed` өгвөл давтагдана
Оюутан бүр өөрийн `random.Random`-тай	Thread хооронд санамсаргүй тооны төлөв хуваалцахгүй, seed-тэй үед давтагдах боломжтой

5.6 Удирдлагын функцүүд

- `pause()` ба `resume()` нь `_run` event-ийг clear ба set хийнэ. Бүх thread-ийн `_sleep()` үүнийг шалгана.
- `stop()` нь `_stop`-ийг set хийж, pause-г тайлж, thread бүрийг `join` хийнэ. Давхар дуудахад алдаа гарахгүй.

- `reset()` нь stop хийж, event queue болон бүх төлвийг шинээр үүсгэнэ. Шинэ тохиргоо өгч болно.

5.7 UI-ийн хэрэгжилт

`Dashboard` класс нь core-ийг `Simulation` объектоор дамжуулан удирдана. Үндсэн ажиллагаа:

- `_poll()` нь 70 мс тутам `sim.drain_events()`-ээр event-үүдийг уншиж `events` буферт нэмнэ.
- `_frame()` нь 33 мс тутам ажиллаж, `Visual speed`-ээр тохируулсан хурдаар (1× үед ойролцоогоор 0.23 секунд тутамд нэг event, нэг frame-д дээд тал нь 4) буферээс event авч `_consume_one()` дуудна.
- `_consume_one()` нь event-ийн snapshot-аас `display_*` хувьсагчуудыг шинэчилж, оюутнуудын байрлал (`Actor`), тоолуур, график, логийг шинэчилнэ.
- Оюутан сандал руу, ТА руу, гарц руу шилжихдээ жигд хөдөлнө (`_advance_actors`). Сандал дүүрэн үед буцсан оюутан улаанаар гарна.
- **STEP** нь нэг event-ийг буферээс авч харуулаад core-ийг pause хийнэ. Буфер хоосон бол core-ийг үргэлжлүүлж дараагийн event-ийг хүлээнэ.
- Event-ийн лог нь төрлөөр шүүх, текстээр хайх, CSV болгон экспортлох боломжтой.

5.8 AI болон гадны материал ашигласан тухай

Төслийн кодын эхний хувилбар болон баримт бичгийн ноорогийг AI туслах (Claude)-ийн тусламжтайгаар боловсруулсан. Баг бүх кодыг уншиж, ажиллуулж, тестээр баталгаажуулсан. Алгоритмын үндсийг Silberschatz-ийн 7-р бүлгээс авсан [1]. Гадны сан ашиглаагүй.

6. Туршилт ба UI дүрслэл

6.1 Юуг өөрчилж турших вэ

Параметр	Өөрчлөх нөлөө	Юу ажиглах
Сандлын тоо	Цөөн бол олон оюутан буцна, олон бол хэн ч буцахгүй	TURNED AWAY тоолуур, дараалал сандлын тооноос хэтрэхгүй эсэх
Оюутны тоо	Олон бол ачаалал нэмэгдэнэ	Дараалал, ТА завгүй байх хугацаа
Ирэх хугацаа	Богино бол олон оюутан нэгэн зэрэг ирнэ	Сандал дүүрэх, ТА унтахгүй байх
Үйлчилгээний хугацаа	Урт бол дараалал өснө	WAITING NOW, SERVED ихсэх хурд

6.2 Удирдлага ба ажиглалтын хэрэгсэл

START, PAUSE, STEP, RESET товчоор симуляцийг удирдаж, STEP-ээр event бүрийг нэг нэгээр нь ажиглана. Visual speed нь зөвхөн дүрслэлийн хурд бөгөөд core thread-ийн хугацаанд нөлөөлөхгүй. Дэлгэц дээрх ТА-ийн төлөв (өнгө, `Z z z`), сандал дээрх оюутан, дээд талын 4 тоолуур, график, event лог нь бүгд core-оос ирсэн event-ээс гарна.

6.3 Сценариуд

11 UI сценарийг дөрвөн төрөлд хуваасан: **хэвийн** (S1, S3), **хязгаар** (S2, S4, S5, S6), **удирдлага** (S7–S9: PAUSE, STEP, RESET) ба **алдаа** (S10, S11: буруу оролт). Дэлгэрэнгүйг 7 ба 8-р хэсэгт үзнэ.

7. Туршилтын арга

7.1 Орчин

Зүйл	Утга
Үйлдлийн систем	macOS 26.6.2
Python	3.14.10 (Tk 9.0)
Төслийн хувилбар	main, commit 198e554
Core	`src/ta_core.py` (thread, Semaphore, Lock)
UI	`src/ui.py` (Tkinter dashboard), Visual speed 1.00×
Огноо	2026.10.28

7.2 Гурван түвшний туршилт

1. **Автомат тест (core):** багийн бичсэн 11 тест (`src/test_ta_core.py`). Тохиргооны шалгалт, invariant (дараалал сандлаас хэтрэхгүй, TA нэг дор нэг оюутанд үйлчилнэ), pause/resume/stop, reset, давхар start.
2. **Edge case тест:** 27 нэмэлт тест (`tests/test_edge_cases.py`). Хязгаарын утга, ер бусын дараалал, synchronization-ы зөв байдал.
3. **UI сценари:** 11 сценарийг UI дээр ажиллуулж, screenshot авч, дэлгэц дээрх тоог хүлээгдэх үр дүнтэй тулгасан (`tests/ui_scenarios.py`).

7.3 Өгөгдөл ба хүлээгдэх үр дүн

Сценари бүрийн хүлээгдэх үр дүнг ажиллуулахаас **өмнө** бичсэн. Симуляци санамсаргүй хугацаа ашигладаг (UI-д seed байхгүй) тул хүлээгдэх үр дүнг тоогоор биш чанарын хэлбэрээр заасан (жишээ нь "хэн ч буцахгүй", "дараалал 20-оос хэтрэхгүй"). UI-ийн тоог бодит core-ийн тоотой харьцуулахын тулд ижил тохиргоог UI-гүйгээр бас ажиллуулсан (`tests/core_scenarios.py`).

UI сценариуд `tests/ui_scenarios.py` скриптээр автоматаар ажилласан: скрипт Dashboard цонхыг нээж, талбаруудад оролтыг бичиж, START, PAUSE, STEP, RESET үйлдлийг хугацаатай дуудаж, тогтоосон мөчид screenshot аваад дэлгэц дээрх тоолуурыг JSON-д хадгалсан.

7.4 Сценарийн тодорхойлолт

ID	Нэр	Төрөл	Оролт	Хүлээгдэх үр дүн
S1	Үндсэн ажиллагаа	Хэвийн	3 / 5 / 1.0 / 1.5	TA эхэнд унтсан. Оюутан ирэхэд сэрнэ, дарааллаар үйлчилнэ

ID	Нэр	Төрөл	Оролт	Хүлээгдэх үр дүн
S2	Сандал дүүрэн	Хязгаар	1 / 10 / 0.3 / 1.5	Оюутнууд буцаж, буцсан тоо нэмэгдэнэ. Дараалал сандлаас хэтрэхгүй
S3	Сандал хангалттай	Хэвийн	10 / 3 / 1.5 / 0.5	Хэн ч буцахгүй, буцсан тоо 0
S4	0 сандал	Хязгаар	0 / 5 / 0.5 / 1.0	Хэн ч үйлчлүүлэхгүй, TA унтсан хэвээр
S5	Нэг оюутан	Хязгаар	1 / 1 / 2.0 / 0.5	Оюутан бүрт TA сэрж, үйлчилсний дараа дахин унтана
S6	Дээд ачаалал	Хязгаар	20 / 50 / 0.2 / 0.5	Гацахгүй. Дараалал 20-оос хэтрэхгүй
S7	PAUSE / RESUME	Удирдлага	3 / 5 / 1.0 / 1.5	PAUSE үед хөдөлгөөн зогсоно. START дарахад үргэлжилнэ
S8	STEP	Удирдлага	3 / 5 / 1.0 / 1.5	STEP нэг дарахад нэг үйл явдал гарна
S9	RESET	Удирдлага	3 / 5 / 1.0 / 1.5	READY төлөв. Бүх тоолуур, event тоо 0
S10	Буруу оролт: хязгаараас хэтэрсэн	Алдаа	25 / 5 / 1.0 / 1.5	Алдааны мессеж гарна, симуляци эхлэхгүй
S11	Буруу оролт: текст	Алдаа	abc / 5 / 1.0 / 1.5	Алдааны мессеж гарна, симуляци эхлэхгүй

Хүснэгт 7.1. Оролт: сандал / оюутан / ирэх хугацаа (сек) / үйлчилгээний хугацаа (сек)

8. Үр дүн

8.1 Автомат тестийн үр дүн

Түвшин	Файл	Тестийн тоо	Үр дүн / Хугацаа
Багийн автомат тест	`src/test_ta_core.py`	11	11/11 тэнцсэн (14.97 сек)
Edge case тест	`tests/test_edge_cases.py`	27	27/27 тэнцсэн (22.8 сек)

Edge case тестүүд гурван бүлэгтэй: тохиргооны хязгаар (9 тест), эхлүүлэх, зогсоох, pause, reset (7 тест), synchronization-ы зөв байдал (11 тест). Үүнд FIFO дараалал, ирэлт бүр суусан эсвэл буцсан байх, дараалал сандлаас хэтрэхгүй, 20 сандал + 50 оюутанд deadlock-гүй, 0 сандалд TA унтсан хэвээр байх зэрэг шалгалт орсон.

8.2 UI сценарийн үр дүн

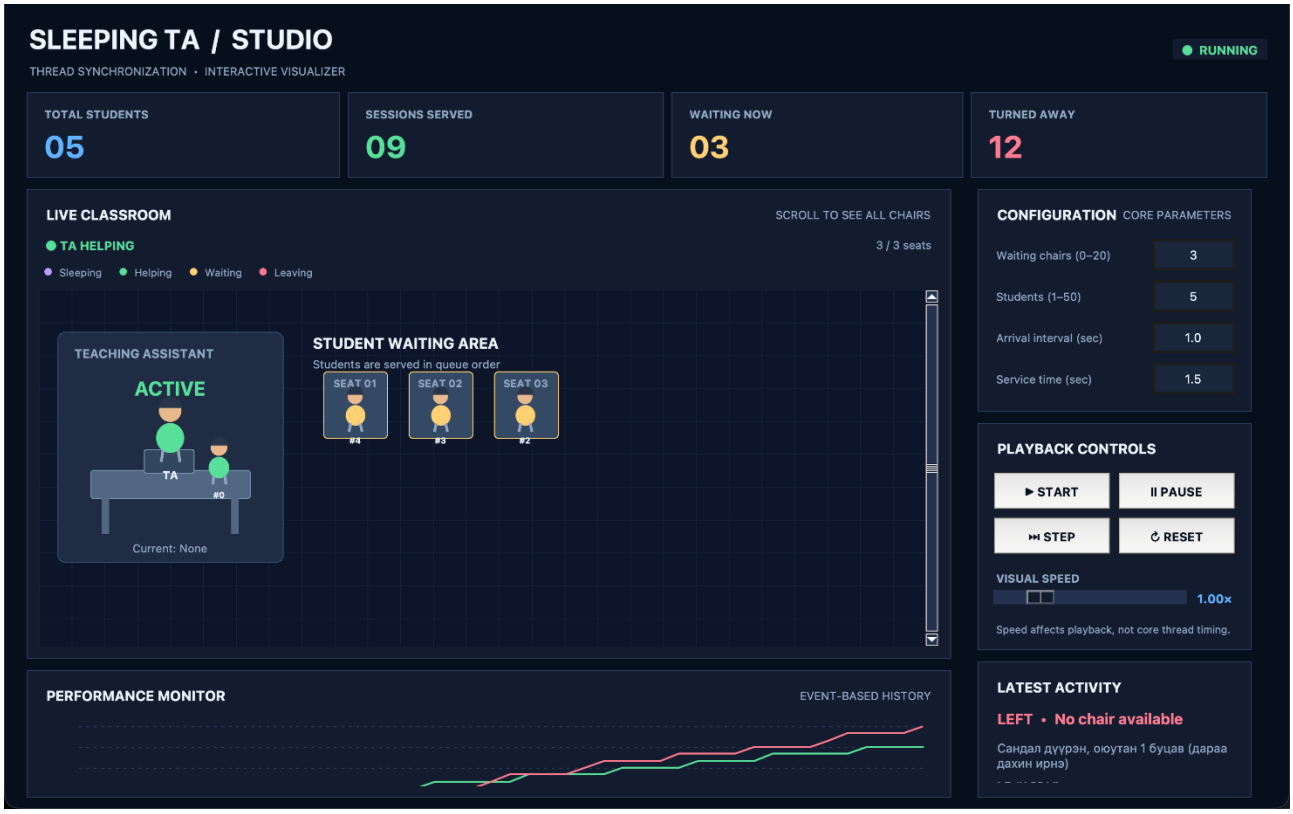
11 сценарийн 11 нь тэнцсэн.

ID	Бодит үр дүн	Үр дүн
S1	Эхлэхээс өмнө TA унтсан. 20 сек-ийн дараа: үйлчлүүлсэн 9, буцсан 12, дараалалд 3, TA HELPING. Core: TA 1 удаа сэрсэн	Тэнцсэн
S2	Буцсан 78, дараалалд 1 (1 сандал дүүрэн), үйлчлүүлсэн 1 (UI)	Тэнцсэн [1]
S3	Үйлчлүүлсэн 21, буцсан 0. Core: TA 10 удаа унтсан	Тэнцсэн
S4	Үйлчлүүлсэн 0, буцсан 64, TA SLEEPING, 0/0 seats	Тэнцсэн
S5	1 оюутан үйлчлүүлсний дараа TA дахин унтсан. 20 сек-ийн дараа үйлчлүүлсэн 6, буцсан 0. Core: TA 6 удаа сэрж, 6 удаа унтсан	Тэнцсэн
S6	Гацаагүй (RUNNING). Дараалалд 20/20 (хэтрээгүй), буцсан 59, үйлчлүүлсэн 1 (UI). Core: 33 үйлчилсэн, дээд дараалал 20	Тэнцсэн [1]
S7	PAUSED үед event тоо 15 дээр 4 сек-ээс илүү өөрчлөгдөөгүй (15 → 15). START-ийн дараа 30 хүртэл үргэлжилсэн	Тэнцсэн

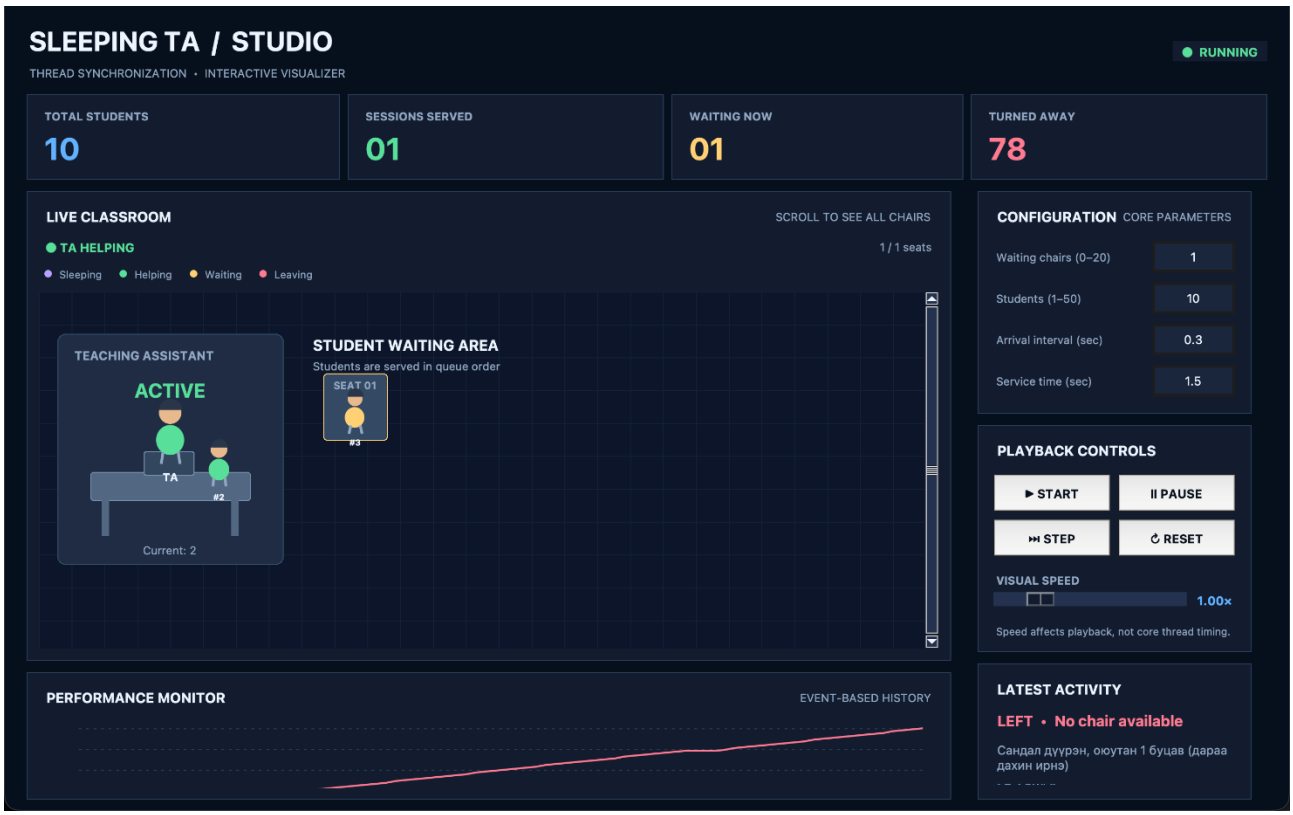
ID	Бодит үр дүн	Үр дүн
S8	Event тоо 6 → 7 → 8 → 9 → 10 → 10. STEP бүр хамгийн ихдээ 1 event нэмсэн. Сүүлийн дарахад event байхгүй тул "WAITING FOR EVENT" гарсан	Тэнцсэн [2]
S9	RESET-ийн өмнө: үйлчлүүлсэн 5, буцсан 5, event 27. Дараа: READY, үйлчлүүлсэн 0, буцсан 0, event 0	Тэнцсэн
S10	INVALID INPUT. Мессеж: "Сандлын тоо 0-20 хооронд бүхэл тоо байна.". Симуляци эхлээгүй	Тэнцсэн
S11	INVALID INPUT. Мессеж: "'Сандлын тоо' нь бүхэл тоо байх ёстой (оруулсан: 'abc').". Симуляци эхлээгүй	Тэнцсэн

Хүснэгт 8.1. UI сценарийн бодит үр дүн

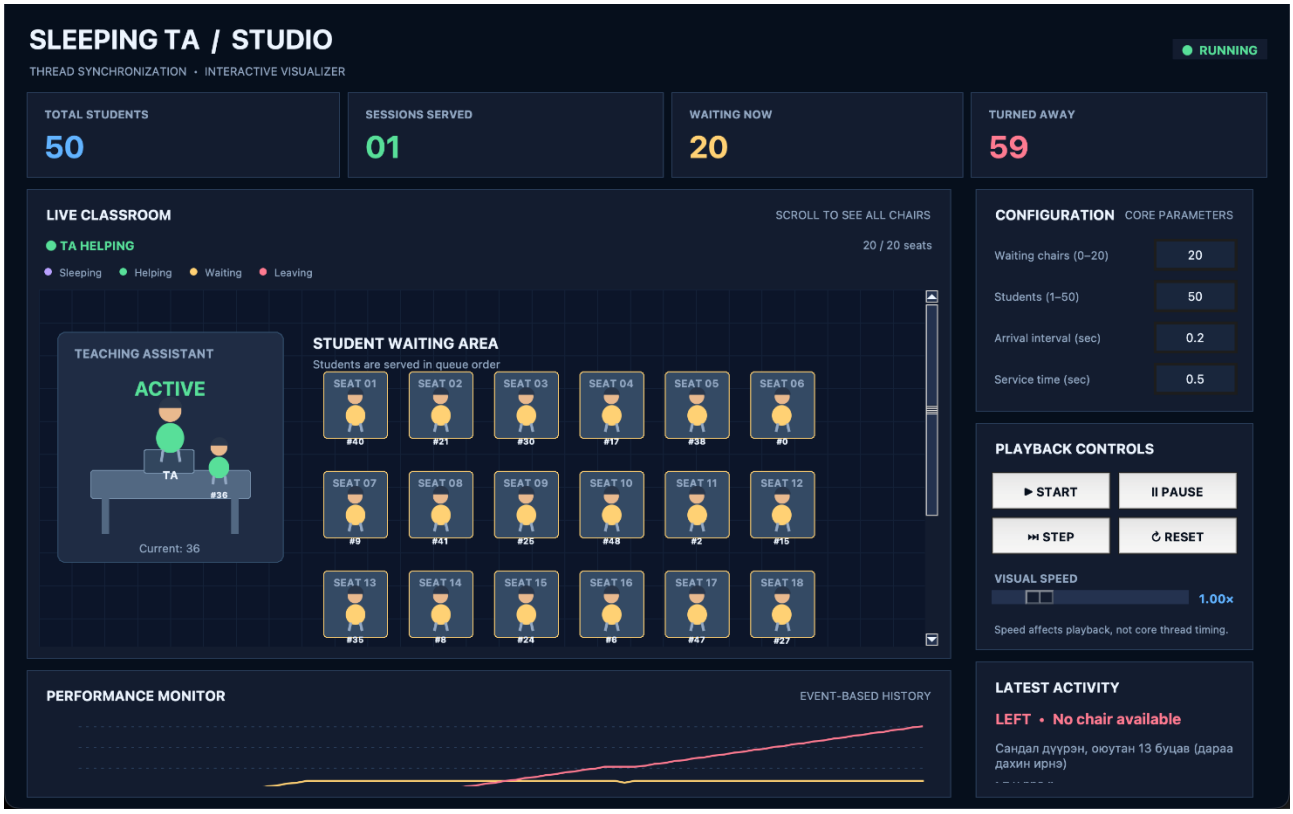
[1] UI-ийн тоо core-оос хоцорч харагдана (8.3, 9.1-ийг үз). [2] STEP нь буфер хоосон үед нэгээс олон event нэг дор үзүүлэх тохиолдол ажиглагдсан (9.2).



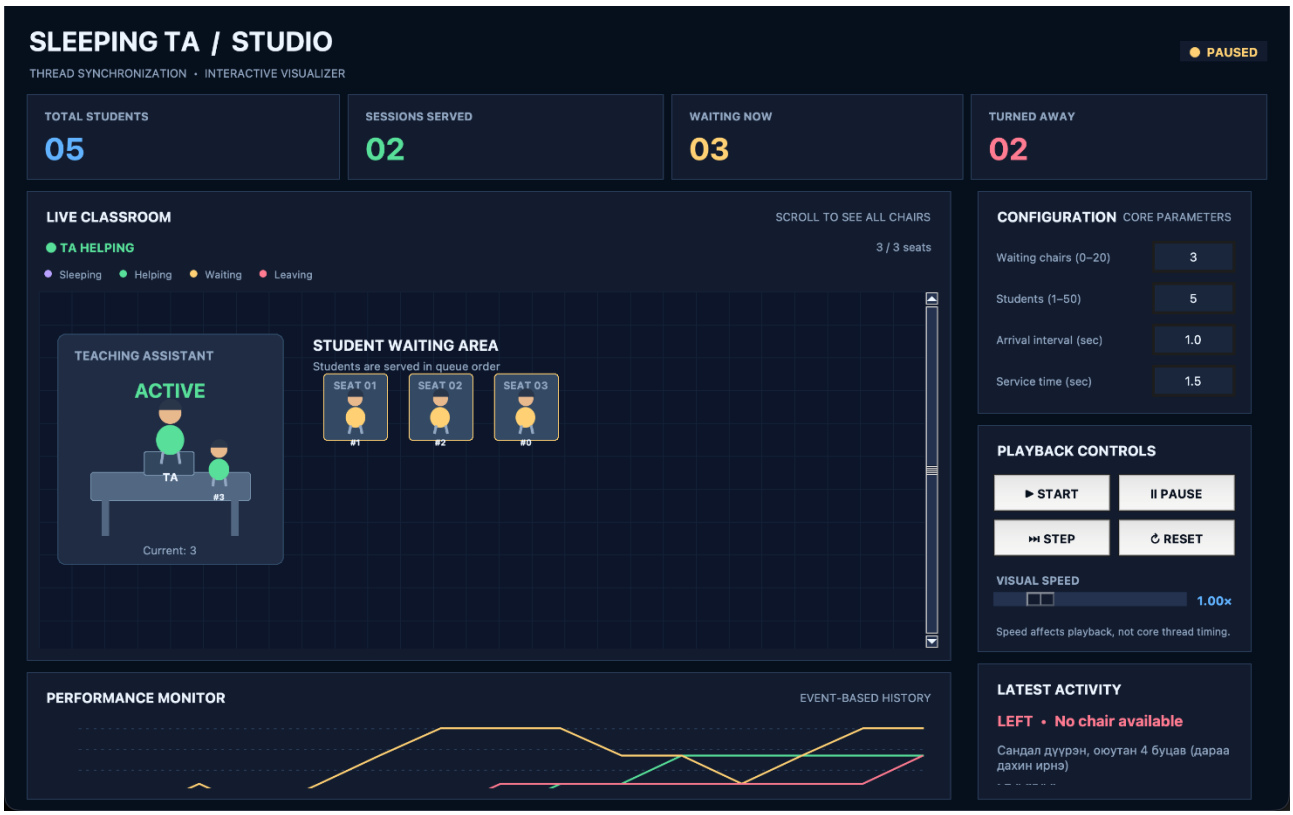
Зураг 8.1. S1: үндсэн ажиллагаа (3 сандал, 5 оюутан)



Зураг 8.2. S2: сандал дүүрэн (1 сандал, 10 оюутан)



Зураг 8.3. S6: дээд ачаалал (20 сандал, 50 оюутан)



Зураг 8.4. S7: PAUSED төлөв

8.3 UI-ийн тоо ба core-ийн тоо

Ижил тохиргоог UI-гүйгээр core дээр шууд ажиллуулсан бодит тоо:

ID	Ролт	Сек	Үйлчилсэн	Буцсан	Суусан	TA сэрсэн	TA унтсан	Дээд дараалал	UI үйлч.	UI буцсан
S1	3/5/1.0/1.5	20	11	11	15	1	0	3	9	12
S2	1/10/0.3/1.5	20	11	434	13	1	0	1	1	78
S3	10/3/1.5/0.5	20	20	0	22	11	10	2	21	0
S4	0/5/0.5/1.0	15	0	120	0	0	0	0	0	64
S5	1/1/2.0/0.5	20	6	0	6	6	6	1	6	0
S6	20/50/0.2/0.5	20	33	2451	54	1	0	20	1	59

Хүснэгт 8.2. Core-ийн бодит тоо (UI-гүй) ба UI дээр харагдсан тоо

S1, S3, S5 (ачаалал бага) дээр UI ба core-ийн тоо ойролцоо байна. Харин буцалт олон болдог S2, S6 дээр UI-ийн тоо core-оос эрс бага байна. Ажиллуулалт бүрд хугацаа санамсаргүй тул ижил тохиргооны хоёр ажиллуулалтын тоо яг таарахгүй (жишээ нь S3-д UI 21, core 20).

9. Хэлэлцүүлэг

9.1 UI-ийн дүрслэл ачаалал ихтэй үед хоцрох

S2 дээр core 20 секундэд 434 оюутныг буцаасан бол UI 78-г харуулсан. S6 дээр core 33 үйлчилсэн, UI 1 харуулсан. Шалтгаан нь UI event-үүдийг тогтмол хурдаар (1× үед ойролцоогоор 0.23 секунд тутамд нэг, буюу секундэд ~4) тоглуулдагт оршино. Харин core ачаалал ихтэй тохиргоонд секундэд хэдэн арван event үүсгэнэ. Event хуримтлагдсаар UI

хоцорно. Энэ нь core-ийн алдаа биш: дээд дараалал сандлын тоотой тэнцүү (S2: 1, S6: 20) байсан нь core-ийн invariant хадгалагдсаныг харуулна.

Цаашдын сайжруулалтаар UI-д event хуримтлагдах үед тоглуулах хурдыг автоматаар нэмэх, эсвэл хоцрол ихтэй үед event-үүдийг нэгтгэх (олон `STUDENT_LEAVES`-ийг нэг болгох) боломжтой. Одоогийн хувилбарт энэ нь мэдэгдэж буй хязгаарлалт бөгөөд бодит тоог `tests/core_scenarios.py`-ээр авч болно.

9.2 STEP товчны ажиллагаа

STEP нь буфер дотор event байвал яг нэгийг үзүүлнэ. Буфер хоосон бол core-ийг үргэлжлүүлж "WAITING FOR EVENT" гэж харуулна. Core-ийн thread-үүд зэрэгцээ ажилладаг тул нэг мөчид хоёр event нэг дор орж ирэх тохиолдол ажиглагдсан. Иймд "нэг дарахад яг нэг event" гэдэг нь дандаа биелэхгүй, "хамгийн ихдээ нэг" гэж илүү зөв.

9.3 Бусад хязгаарлалт

- **Буруу тохиргоотой `reset(new_config)`:** reset нь эхлээд stop хийж, дараа нь тохиргоог шалгадаг тул шалгалт алдаатай бол симуляци зогссон хэвээр үлддэг. UI тохиргоог урьдчилан шалгадаг тул хэрэглэгчид тохиолдохгүй. Reset-д эхлээд тохиргоог шалгаад дараа нь stop хийвэл энэ арилна.
- **UI-д seed талбар байхгүй:** core seed дэмждэг ч UI-д талбар байхгүй тул нэг сценарийг яг ижил тоогоор давтах боломжгүй. Тиймээс хүлээгдэх үр дүнг чанарын хэлбэрээр бичсэн.
- **Платформ:** туршилтыг зөвхөн macOS дээр хийсэн. `ui_scenarios.py` нь macOS-ийн `screencapture` ашигладаг. Core ба автомат тест нь платформоос хамаарахгүй.
- **Туршаагүй зүйл:** олон цагийн урт хугацааны ажиллагаа болон Windows дээрх UI-г туршаагүй.

9.4 Synchronization-ы зөв байдлын тайлбар

Бүх тохиргоонд дараах шинж биелсэн: (1) дараалал сандлын тооноос хэтрээгүй, давхцаагүй; (2) TA нэг дор нэг оюутанд л үйлчилсэн; (3) эхэлж суусан оюутан эхэлж үйлчлүүлсэн; (4) 20 сандал, 50 оюутанд гацаагүй, тест дууссаны дараа thread үлдээгүй; (5) 0 сандалд TA унтсан хэвээр, хэн ч үйлчлүүлээгүй. Эдгээр нь mutex-ээр хамгаалсан дараалал, semaphore-оор унтах/сэрэх шийдэл зөв ажиллаж буйг харуулна.

10. Дүгнэлт

Энэ бие даалтаар Sleeping Teaching Assistant бодлогыг Python-ийн `threading` сан ашиглан бодитоор хэрэгжүүллээ. TA болон оюутан бүр тусдаа thread, хүлээлгийн дараалыг mutex, TA-ийн унтах ба сэрэх үйлдлийг semaphore-оор зохицуулсан. Оюутан бүрт хувийн semaphore өгснөөр FIFO дараалал бодитой, UI-тай нийцтэй болсон.

Core нь UI-аас хамааралгүй бөгөөд event queue-ээр дамжуулан Tkinter dashboard-д мэдээллээ өгдөг. Dashboard нь TA-ийн төлөв, хүлээлгийн сандал, оюутны хөдөлгөөн, тоолуур, график,

event түүхийг бодит thread-үүдийн төлөвөөр харуулж, START, PAUSE, STEP, RESET удирдлага, шүүлт, хайлт, CSV экспорттой.

Хэрэгжилтийг 11 автомат тест, 27 edge case тест, 11 UI сценариар баталгаажуулсан бөгөөд бүгд тэнцсэн. Гол хязгаарлалт нь ачаалал ихтэй тохиргоонд UI-ийн дүрслэл core-ийн бодит тооноос хоцордог явдал бөгөөд core-ийн зөв байдалд нөлөөлөхгүй. Цаашид event нэгтгэх, хурдыг автоматаар тохируулах, UI-д seed талбар нэмэх, Windows дээр туршихыг санал болгоно.

Түвшин	Файл	Тестийн тоо	Үр дүн / Хугацаа
Багийн автомат тест	`src/test_ta_core.py`	11	11/11 тэнцсэн (14.97 сек)
Edge case тест	`tests/test_edge_cases.py`	27	27/27 тэнцсэн (22.8 сек)
Core сценари	`tests/core_scenarios.py`	6 сценари (S1–S6)	JSON-д хадгалагдсан (~2 мин)
UI сценари	`tests/ui_scenarios.py`	11 сценари (S1–S11)	11/11 тэнцсэн (~3.5 мин)
Нийт	Бүх файлууд	38 автомат + 11 UI	Бүгд тэнцсэн

Ашигласан материал

[1] Silberschatz, A., Galvin, P. B., Gagne, G. Operating System Concepts. 10th Edition. Wiley, 2018. Chapter 6 (Synchronization Tools), Chapter 7 (Synchronization Examples), Programming Project 10: The Sleeping Teaching Assistant.

[2] Python Software Foundation. Python documentation: threading, queue, tkinter, unittest, dataclasses. https://docs.python.org/3/

[3] Багийн GitHub repository: https://github.com/B242270001/II-4-10_SleepingTeachingAssistant

[4] II-4-10 баг. Туршилт, Үр дүн (testing_results.pdf). 2026.10.28.

[5] II-4-10 баг. Хэрэглэгчийн болон Operational гарын авлага (II-4-10_Manual.pdf).

Хавсралт А. Ажлын төлөвлөгөө

Талбар	Мэдээлэл
Хуваарилалтын код	II-4-10
Төсөл	The Sleeping Teaching Assistant
Repository	https://github.com/B242270001/II-4-10_SleepingTeachingAssistant
Хэл / сан / орчин	Python 3.14, Tk 9.0, зөвхөн стандарт сан. macOS дээр туршсан
Үндсэн дүрслэл	Queue animation: хүлээлгийн дараалал ба TA-ийн төлөвийг хөдөлгөөнт, алхамчилсан байдлаар харуулна

Гишүүн	Хариуцсан ажил	Commit / файл
А.Төгөлдөр (B242270001)	Core: `ta_core.py`, core-ийн тест	`src/ta_core.py`, `src/test_ta_core.py`
Ц.Цогтжаргал (B241940005)	Баримт: GitHub repo/README/release tag, Manual.pdf (хэрэглэгч ба operational), Report-ийн нүүр, удиртгал, дүгнэлт, Teams пост	README.md, docs/
Д.Отгон-Эрдэнэ (B222270083)	Туршилт: сценари тодорхойлж, screenshot авч, үр дүнгийн хүснэгт бөглөх, edge case нэмэх	tests/, Report "Туршилт, Үр дүн"
О.Цэрэнлхавга (B232270113)	Tkinter UI: TA, сандлууд, оюутнуудын зураг, өнгө, legend, event log	src/ui.py
Г.Түвшинсайхан (B242270085)	Тохиргооны самбар (4 оролт, Start/Pause/Step/Reset, validation мессеж) болон step/animation горимын логик	src/ui.py

Тест	Оролт / нөхцөл	Хүлээгдсэн үр дүн
1	3 сандал, 5 оюутан, ирэх 1.0, үйлчилгээ 1.5 (S1)	TA эхэнд унтсан, оюутан ирэхэд сэрнэ, дарааллаар үйлчилнэ
2	1 сандал, 10 оюутан, ирэх 0.3 (S2)	Оюутнууд буцна, дараалал 1-ээс хэтрэхгүй
3	10 сандал, 3 оюутан (S3)	Хэн ч буцахгүй
4	0 сандал (S4)	Хэн ч үйлчлүүлэхгүй, TA унтсан хэвээр
5	20 сандал, 50 оюутан (S6)	Гацахгүй, дараалал 20-оос хэтрэхгүй